

Unrolled List

–Memory Reference and Allocation

64 – bit architecture:

```
struct Node {  
    int numElements;  
    int elements[MAX_ELEMENTS];  
    struct Node* next;  
};
```

When a compiler processes the assignment `node->numElements = 4;`, it executes the following logical sequence:

- It retrieves the base memory address stored in the `node` pointer, representing where the struct begins in memory (hypothetically `0x1000`).
- It references the struct's blueprint to determine the offset of `numElements`. Because `numElements` is the first declared member, it physically begins at the exact start of the struct block, making its offset `0`.
- It computes the target destination by adding the member's offset to the base address (`0x1000 + 0 = 0x1000`).
- It writes the value `4` starting at that computed address (`0x1000`). Because an integer occupies 4 bytes on a 64-bit architecture, the value is distributed across the memory addresses `0x1000` through `0x1003` as a 32-bit binary sequence.

→ $baseaddress + offset = 0x1000 + 0[bytes] = \text{Where } numElement \text{ is stored.}$

Now, note: **numElements** is integer data type and 4 bytes size for 64 – bit architecture .

$$0x1000_{16} = 1 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 1 \times 4096$$

$$= 4096_{10} = \frac{4096_{10}}{4 \text{ bytes}} = \frac{4096_{10}}{4_{10}} = 1024_{10} \text{ [Hence, divisible by 4 bytes]}$$

∴ Valid address to store Integer data.

Hence allocated addresses are :

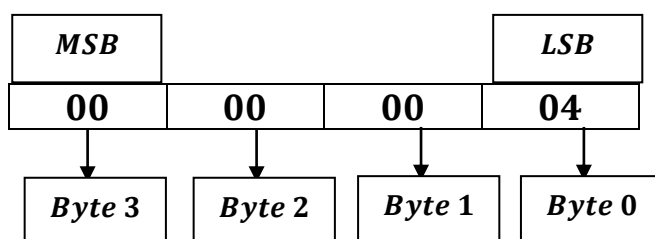
0x1000

0x1001

0x1002

0x1003

Hence , offset: 0 – 3.



0x1000 = 0x04 → Least Significant Byte

0x1001 = 0x00

0x1002 = 0x00

0x1003 = 0x00 → Most Significant Byte

As we know everything is stored in Memory address at binary format.

0x1000 = 0000 0100[8 bits]

0x1001 = 0000 0000[8 bits]

0x1002 = 0000 0000[8 bits]

0x1003 = 0000 0000[8 bits]

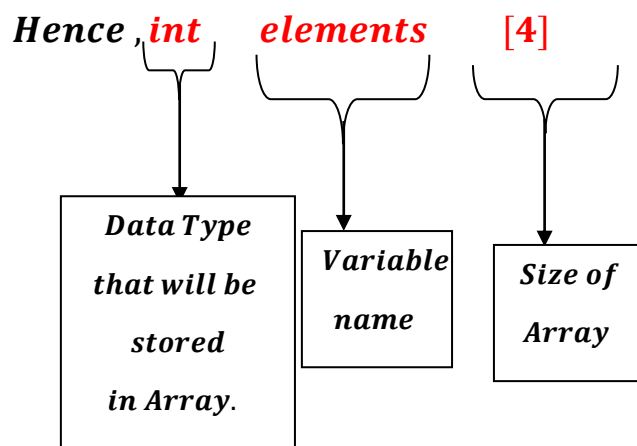
[8 × 4 = 32 bits]

Next , compiler sees:

```
int elements[MAX_ELEMENTS];
```

And ,

```
#define MAX_ELEMENTS 4
```



Therefore datatype is integer i. e. 4 bytes :

According to the provided document, when the compiler sees `int elements [4];`, it thinks:

- "Get the base memory address stored in the pointer node." (e.g., `0x1000`)
- "Look up the definition of the `Node` struct. The `numElements` integer occupies the first 4 bytes, meaning the offset for the `elements` array is 4."
- "Calculate the starting address: `base address + offset => 0x1000 + 4 => 0x1004`."
- "Recognize that `elements[4]` requires space for 4 integers. Since each integer is 4 bytes, reference or allocate a 16-byte memory block starting at `0x1004` (spanning `0x1004` through `0x1013`)."

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1000	<i>E.g.: numElements = 1, 2 ... etc.</i>	<i>numElements[0x1000 – 0x1003] 4 bytes for int</i>
0x1004	<i>E.g.: elements = {15, 42, ... etc.}</i>	<i>elements[0x1004 – 0x1013] 16 bytes for int array</i>

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1004	<i>elements[0]</i>	<i>elements[0] → [0x1004 – 0x1007] 4 bytes for int</i>
0x1008	<i>elements[1]</i>	<i>elements[1] → [0x1008 – 0x100b] 4 bytes for int</i>

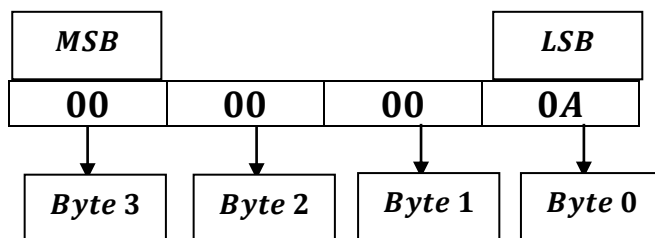
0x100c	<i>elements[2]</i>	<i>elements[2] →</i> <i>[0x100c – 0x100f]</i> <i>4 bytes for int</i>
0x1010	<i>elements[3]</i>	<i>elements[3] →</i> <i>[0x1010 – 0x1013]</i> <i>4 bytes for int</i>

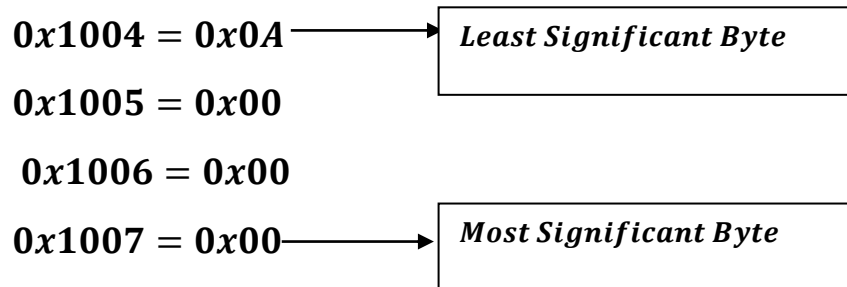
	<i>Address</i>	<i>Values</i>
<i>Elements</i> [0]	0x1004	10
<i>Elements</i> [1]	0x1008	20
<i>Elements</i> [2]	0x100c	30
<i>Elements</i> [3]	0x1010	40

Elements[0] = 10

0x1004 is divisible by 4 bytes , hence valid address to start with.

offset: 4 – 7.





As we know everything is stored in Memory address at binary format.

$0x1004 = 0000\ 1010[8\ bits]$

$0x1005 = 0000\ 0000[8\ bits]$

$0x1006 = 0000\ 0000[8\ bits]$

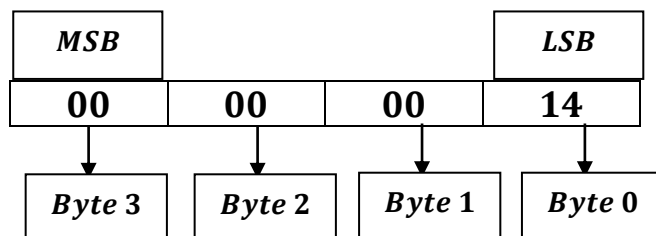
$0x1007 = 0000\ 0000[8\ bits]$

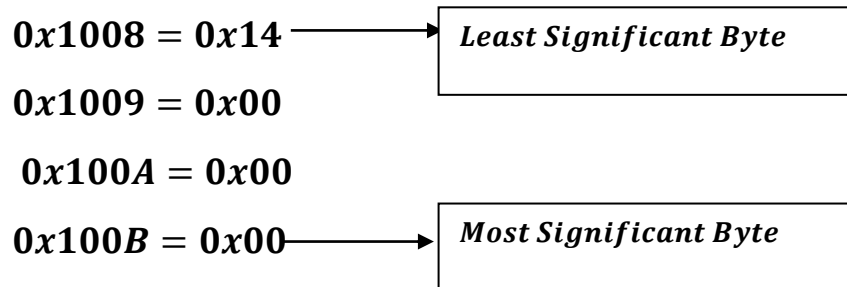
$[8 \times 4 = 32\ bits]$

Elements[1] = 20

0x1008 is divisible by 4 bytes , hence valid address to start with.

offset: 8 – 11.





As we know everything is stored in Memory address at binary format.

$0x1008 = 0001\ 0100[8\ bits]$

$0x1009 = 0000\ 0000[8\ bits]$

$0x100A = 0000\ 0000[8\ bits]$

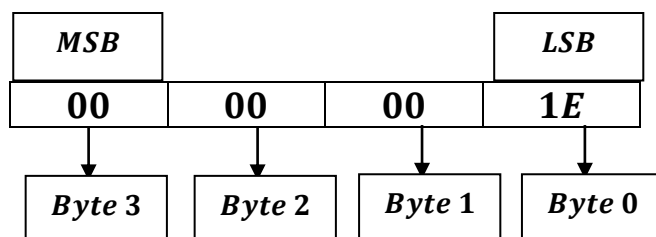
$0x100B = 0000\ 0000[8\ bits]$

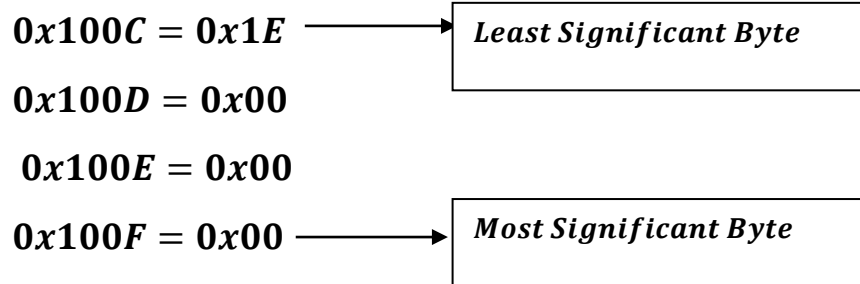
$[8 \times 4 = 32\ bits]$

Elements[2] = 30

0x100C is divisible by 4 bytes , hence valid address to start with.

offset: 12 – 15.





As we know everything is stored in Memory address at binary format.

$0x100C = 0001\ 1110[8\ bits]$

$0x100D = 0000\ 0000[8\ bits]$

$0x100E = 0000\ 0000[8\ bits]$

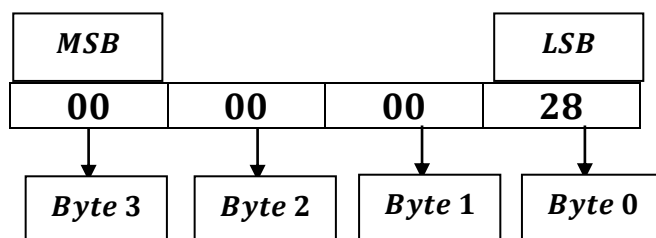
$0x100F = 0000\ 0000[8\ bits]$

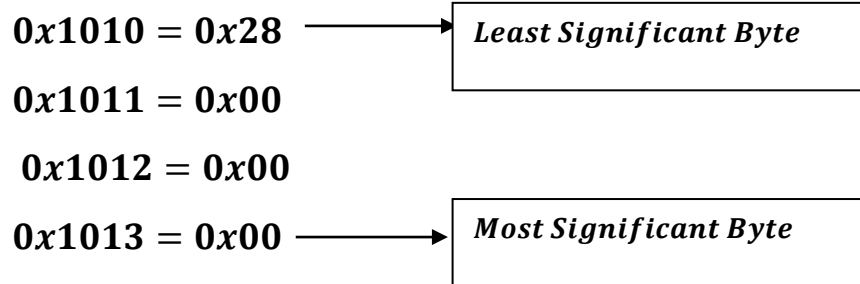
$[8 \times 4 = 32\ bits]$

Elements[3] = 40

0x1010 is divisible by 4 bytes , hence valid address to start with.

offset: 15 – 18.





As we know everything is stored in Memory address at binary format.

0x1010 = 0010 1000[8 bits]

0x1011 = 0000 0000[8 bits]

0x1012 = 0000 0000[8 bits]

0x1013 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Padding

Compiler Logic for Struct Padding

- "Evaluate the current memory offset after the elements array. Because elements[3] concludes at 0x1013, the next available consecutive memory address is 0x1014."
- "Look up the alignment rules for the 64-bit architecture specified in the struct's blueprint. The upcoming member, struct Node* next, is a memory pointer. On a 64-bit system, pointers require 8 bytes and must be aligned to a memory address divisible by 8."

- "Calculate the alignment: 0x1014 (decimal 4116) is not evenly divisible by 8. Determine that 4 bytes of padding are necessary to reach the next divisible boundary (0x1018)."
- "Insert a 4-byte padding block starting at 0x1014 (spanning 0x1014 through 0x1017) to satisfy structural alignment, establishing the correct new offset at 24 bytes."

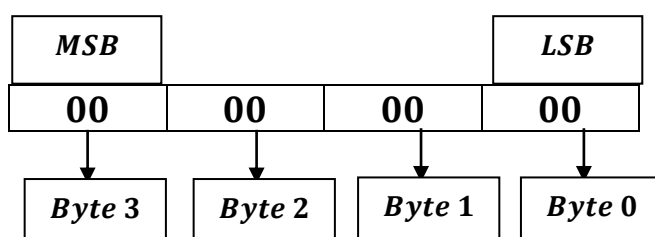
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
		<i>padding → [0x1014 – 0x1017] 4 bytes for padding</i>

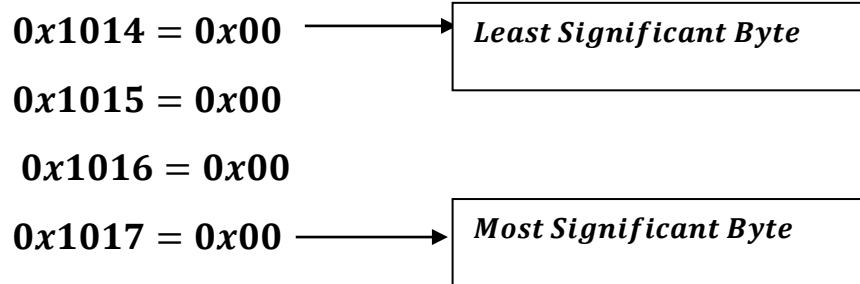
offset: 19 – 22.

As, next is 8 byte pointer and 0x1014 divided by 8 byte*

$$0x1014 = \frac{4116_{10}}{8 \text{ byte}} = 514 \text{ remainder } 4, \text{ hence not divisible,}$$

we need address of 8 byte allignment, therefore we need 4 byte padding.





As we know everything is stored in Memory address at binary format.

0x1014 = 0000 0000[8 bits]

0x1015 = 0000 0000[8 bits]

0x1016 = 0000 0000[8 bits]

0x1017 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

** next pointer*

Compiler Logic for struct Node* next

- "Get the base memory address stored in the pointer node."
(e.g., 0x1000)
- "Look up the definition of the Node struct. After accounting for numElements (4 bytes), elements[4] (16 bytes), and the inserted architectural padding (4 bytes), the resulting offset for the next pointer is 24 bytes (0x18 in hexadecimal)."

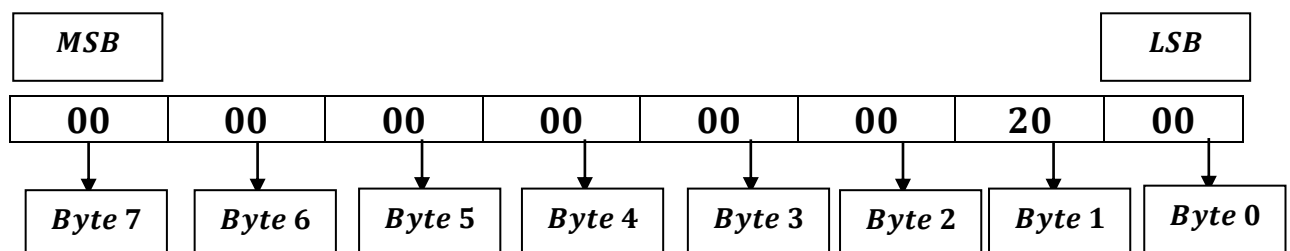
- "Calculate the starting address: base address + offset => 0x1000 + 0x18 => 0x1018."
- "Recognize that struct Node* next is a pointer operating on a 64-bit architecture. Reference or allocate an 8-byte memory block starting at 0x1018 (spanning 0x1018 through 0x101F)."

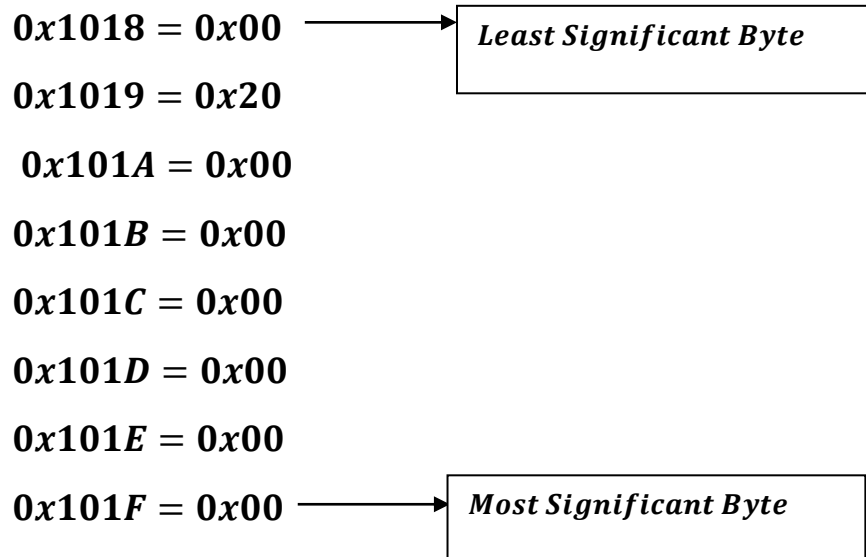
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1018	<i>E.g.: Address: 0x2000</i>	* next → [0x1018 – 0x101F] 8 bytes for pointer variable

offset: 23 – 30.

$$0x1018_{16} = 4120_{10} \text{ divided by 8 bytes} = \frac{4120_{10}}{8_{10}} = 515 \text{ and}$$

Remainder = 0, hence valid memory address to start with.





As we know everything is stored in Memory address at binary format.

$0x1018 = 0000\ 0000[8\ bits]$
 $0x1019 = 0010\ 0000[8\ bits]$
 $0x101A = 0000\ 0000[8\ bits]$
 $0x101B = 0000\ 0000[8\ bits]$
 $0x101C = 0000\ 0000[8\ bits]$
 $0x101D = 0000\ 0000[8\ bits]$
 $0x101E = 0000\ 0000[8\ bits]$
 $0x101F = 0000\ 0000[8\ bits]$

$[8 \times 8\ bits = 64\ bits]$

For 32 – bit architecture ,

Pointer Variable → 4 bytes [32 bit]

Integer Variable → 4 bytes [32 bit]

numElements = 4

0x1000 = 0x04

0x1001 = 0x00

0x1002 = 0x00

0x1003 = 0x00

in Bits :

0x1000 = 0000 0100[8 bits]

0x1001 = 0000 0000[8 bits]

0x1002 = 0000 0000[8 bits]

0x1003 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[0] = 10

0x1004 = 0x0A

0x1005 = 0x00

0x1006 = 0x00

0x1007 = 0x00

in Bits :

0x1004 = 0000 1010[8 bits]

0x1005 = 0000 0000[8 bits]

0x1006 = 0000 0000[8 bits]

0x1007 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[1] = 20

0x1008 = 0x14

0x1009 = 0x00

0x100A = 0x00

0x100B = 0x00

in Bits :

0x1008 = 0001 0100[8 bits]

0x1009 = 0000 0000[8 bits]

0x100A = 0000 0000[8 bits]

0x100B = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[2] = 30

$0x100C = 0x1E$

$0x100D = 0x00$

$0x100E = 0x00$

$0x100F = 0x00$

in Bits:

$0x100C = 0001\ 1110[8\ bits]$

$0x100D = 0000\ 0000[8\ bits]$

$0x100E = 0000\ 0000[8\ bits]$

$0x100F = 0000\ 0000[8\ bits]$

$[8 \times 4 = 32\ bits]$

Elements[3] = 40

$0x1010 = 0x28$

$0x1011 = 0x00$

$0x1012 = 0x00$

$0x1013 = 0x00$

in Bits:

0x1010 = 0010 1000[8 bits]

0x1011 = 0000 0000[8 bits]

0x1012 = 0000 0000[8 bits]

0x1013 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

** next pointer*

0x1014 = 0x00

0x1015 = 0x20

0x1016 = 0x00

0x1017 = 0x00

in Bits:

0x1014 = 0000 0000[8 bits]

0x1015 = 0010 0000[8 bits]

0x1016 = 0000 0000[8 bits]

0x1017 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

On a typical 32 – bit architecture, a 4 – byte pointer occupies 32 bits. A 4 – byte integer also occupies 32 bits. If the integer is followed by a 4 – byte – aligned pointer, no padding is normally required because the 4 – byte integer already ends at an offset that is aligned to 4 bytes. Thus, the pointer can immediately follow the integer while maintaining 4 – byte alignment.
